Assignment - 2

① what is three address code? mention its types. How would you implement the three address statements. Explain with examples.

TAC :- It is an intermediate representation where each instruction typically has one operator and at most three operands.

* If you want to write TAC for the given expression follow the step :-

1. instructions are very simple
2. LHS is the target value and RHS is has the atmost two sources.
3. RHS part can be constant value / variable
4. TAC is a generic form.

TAC types

1. Quadruple :- It is a structure which consists of 4 fields namely operator, arg1, arg2 and result.

Scanned with OKEN Scanner

* op denotes the operator and arg1 & arg2 denotes the two operands and result is used to store the result of the expression

2. Triples :-

This representation doesn't make use of extra temporary variable to represent a single operation instead when a reference to another triple's value is needed, a pointer to that triple is used.

* So, it consist of only three fields namely op, arg1 & arg2.

3. Indirect triples :-

These representation makes use of pointer to the listing of all references to computations which is made separately & stored.

* It is similar in utility as compared to quadruple representation but requires

less space than it. Temporaries are implicit & easier to rearrange code.

Eg:- $a + b * c - d / (b * c)$

$t_1 = b * c$

$t_2 = a + b$

$t_3 = c - d$

$t_4 = t_2 * t_3$

$t_5 = t_4 / t_1$

Quadruple :-

op	arg1	arg2	result
*	b	c	t_1
+	a	b	t_2
-	c	d	t_3
*	t_2	t_3	t_4
/	t_4	t_1	t_5

Scanned with OKEN Scanner

Triples :-

	operator	arg1	arg 2
0	+	b	c
1	+	a	b
2	-	c	d
3	*	1	2
4	/	3	0

Indirect triples :-

op	arg1	arg2
+	b	c
+	a	b
-	c	d
*	101	102
/	103	100

4

Scanned with OKEN Scanner

(2) Construct a quadruple, triple & indirect triple for the stmt $a+a*(b-c)+(b-c)*d$.

Given expression

$$a+a*(b-c)+(b-c)*d$$

$t_1 = a+a$

$t_2 = b-c$

$t_3 = t_1 * t_2$

$t_4 = t_2 * d$

$t_5 = t_3 + t_4$

Quadruple

op	arg1	arg2	result
+	a	a	t_1
-	b	c	t_2
*	t_1	t_2	t_3
*	t_2	d	t_4
+	t_3	t_4	t_5

Scanned with OKEN Scanner

Triple

op	arg1	arg2
+	a	a
-	b	c
*	0	1
+	1	d
+	2	3

Indirect Triple

op	arg1	arg2
+	a	a
-	b	c
*	100	101
*	101	d
+	102	103

6

Scanned with OKEN Scanner

(3) Describe the issues ... generator.

* A code generator is a crucial part of a compiler that converts the intermediate representation of source code into machine readable instructions.

* It's main task is to produce the correct & efficient code that can be executed by a computer.

* The design of the code generator should ensure that it is easy to implement, test, maintain.

Several issues

1. input to code generator

* It comes from the intermediate code generated by the compiler's front-end.

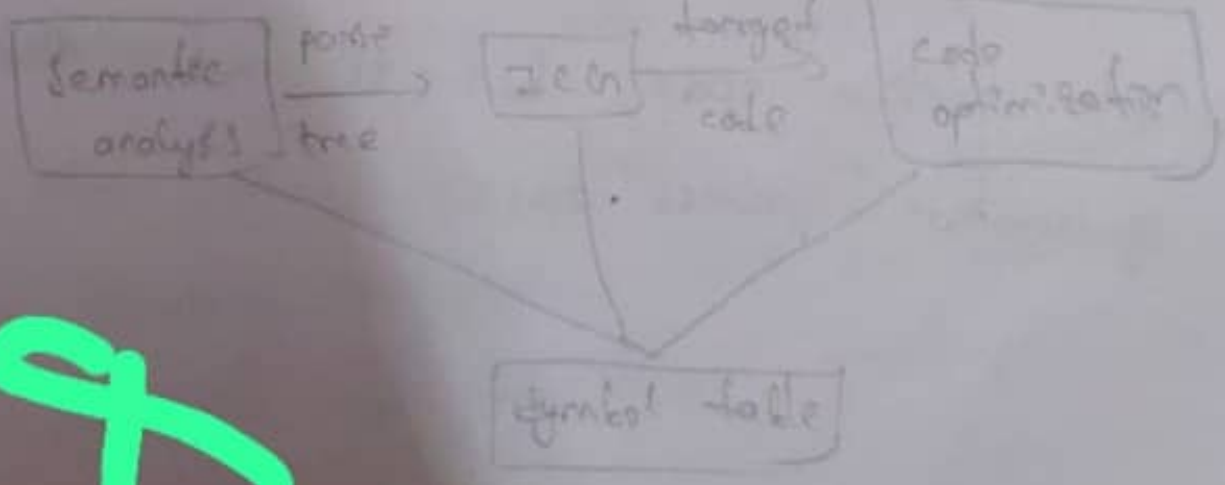
Scanned with OKEN Scanner

* this intermediate code is usually a higher-level representation of the pgm like triples, quadruples or abstract syntax trees.

* Along with this intermediate code, the CG also uses information from the symbol table, which holds the addresses of variables & other data objects.

* one key challenge here is that the input must be free from syntactic & semantic errors,

* CG assumes that proper type-checking & other errors checks have already been handled by the front-end. Handling the I/P correctly is crucial for generating the correct target code.



2. Target program

* It is the final o/p of the CG, which can be in the form of absolute machine language, relocatable m/c, assembly language.

absolute m/c :- is easy to execute but lacks flexibility because it is bound to specific memory locations.

Relocatable m/c :- allows parts of the pgm to be moved around in memory, making it suitable for linking multiple modules, but it requires a linking loader & has some overhead.

Assembly Language :- is symbolic & needs an additional step (an assembler) to convert it into machine code, but it makes the code generation process easier.

Scanned with OKEN Scanner

3. memory management :-

* If involves mapping variables names to their corresponding memory locations.

* the cg works closely with the front end to access the symbol table, where memory addresses for variables are stored.

* this requires careful handling of variable storage, particularly for dynamically allocated objects or large data structures, such as arrays or objects in object-oriented language.

4. Instruction selection :-

* It is the process of choosing the most suitable machine instructions to translate intermediate code into executable code.

* the goal is to optimize the generated code by selecting instructions that are efficient & appropriate for the target machine.

* If the right instructions are not selected, the resulting code can be inefficient & slow.

Eg:- the respective three-address stmts would be translated into the Lattice code sequence

$x = y + z$ → TAC

↓ into

Assembly code

```
MOV  y, R0
ADD  z, R0
move R0, x
```

5. Register allocation issues

* Efficient use of registers is important because registers are faster than memory & utilizing them effectively can significantly improve pgm performance.

* This involves two stages:-

1. register allocation
2. " assignment

register allocation :- It is selecting which variables will reside in the registers at each point in the pgm.

register Assignment :- Assigning specific registers to those variables selected in register allocation

6. Evaluation order :-

* It refers to the sequence in which expressions are evaluated in the generated code.

* This order can significantly affect the efficiency of the pgm.

* It is often a complex problem in some cases & it may requires sophisticated algorithms to find the optimal solution.

12

Scanned with OKEN Scanner

④ Explain peephole optimization

example.

peephole optimization:- this technique is applied to improve the performance of program by examining a short sequence of instructions in a window (peephole) & replace the instructions by a faster or short sequence of instructions.

* the peephole is machine-dependent optimization.

peephole optimization techniques:-

1. Redundant instruction elimination:- removes instructions that are unnecessary.

Eg:- consider one instructions

```
mov R0, a
mov a, R0
```

we can eliminate 2nd instruction, since 'a' is already in R0.

```
mov R0, a
```

13

Scanned with OKEN Scanner

2. Removal of unreachable code :- Eliminate the stmts which are unreachable i.e, never executed.

Eg1:-

```
i=0;                    | i=0;
if(i==1)                |
{                       |
  Sum=0;                |
}                       |
```

Eg2:-

```
int add (int a, int b)
{
    int c=a+b;
    return c;
    printf("%d", c);
}
```

14

3. flow of control optimizations :- unnecessary jumps can be eliminated.

Eg:

```
    goto L1
    ...
L1: goto L2
    ...
L2: goto L3
    ...
L3: mov a, R0
```

multiple jumps can make the code inefficient.

Above code can be replaced by

```
        goto L3
        ....
L1 :    goto L3
        ....
L2 :    goto L3
        ....
L3 :    mov a, Ro
```

4. Algebraic simplification :- uses algebraic laws to simplify or reorder instructions.

Eg:- x = x + 0 (or) x = x * 1

one above stmts can be eliminated because by executing those stmts the result 'x' doesn't changes.

Eg:- a = x ^ 2 replaced with a = x * x

Eg:- b = y / 8 " " b : y >> 3

Scanned with OKEN Scanner

5. constant folding:- Evaluates expressions with constant operands at compile time, replacing them with their result.

Eg:- $r2 := 3 * 2$ becomes $r2 := 6$

6. Strength reduction:- replace a costly operation with cheaper one.

Eg:- $r1 := r2 * 2$ can be replaced with

$r1 := r2 << 1$

(5) what is a basic block? How to construct a basic block.

Basic block:- It is a sequence of consecutive instructions with a single entry & single exit point, meaning control flow enters at the beginning & leaves at the end without any branches in the middle.

Scanned with OKEN Scanner

* A basic block is a linear sequence of instructions.

* It has exactly one entry point & one exit point.

* control flow enters at the beginning & proceeds through the instructions sequentially until it reaches the end, without any branching in or out except at the very start and end.

* This property makes them useful for compiler optimizations, as you can analyze & transform a basic block independently.

How to construct a basic block

1. Identify the "leaders": find all the statements that will begin a new basic block. The leaders are :-

* The very first instruction in the code.

* Any instruction that is the target of a goto or other jump instruction (conditional or unconditional)

* Any instruction that immediately follows a goto or jump instruction.

2. Define the blocks:- once the leaders are identified, create a basic block for each one.

* Each block starts with a leader stmt.

* The block contains all instructions from that leader up to, but not including, the next leader stmt.

* If there are no more leaders, the block continues to the end of the program.

3. partition the code:- Go through the sequence of three-address code instructions and assign each one to exactly one basic block based on the leaders you have identified.

⑥ Explain various storage allocation strategies with an example.

* A compiler is a program that converts HLL into LLL like machine learning.

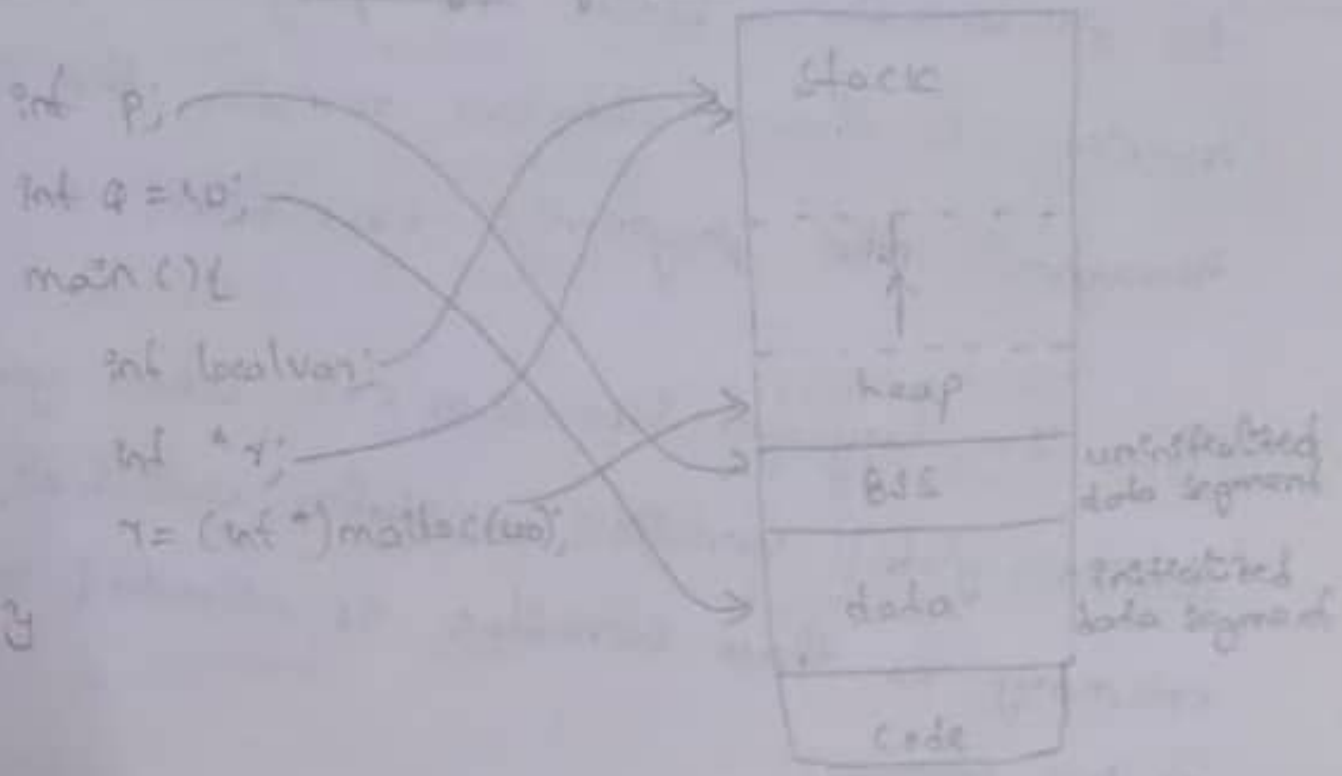
* It's also responsible for organizing the memory layout of a program by allocating space for global & local variables, constants and dynamic data structures.

* A program begins execution, it is under the control of the operating system, which sets up the environment by allocating space for the whole process.

* compiler is responsible for deciding which variable gets which part of the memory.
for eg:- the compiler needs to decide whether variables are best placed in the stack or heap or data segment.

Scanned with OKEN Scanner

memory layout of C program



There are mainly 3 types of storage allocation strategies which compiler uses :-

1. Static allocation :-
   storage

* It is the simplest form of memory allocation.

* In this, the memory for variables is allocated at compile time.

20

* The compiler determines the memory address for all variables before the program starts running. These addresses remain the same throughout the program's execution.

Eg:- In languages like C or C++, when you declare global variables or static variables, the memory for these variables is allocated using static storage.

```
int number = 1; //Global
static int digit = 1;
```

Advantage

1. Easy to understand
2. process is straightforward
3. memory allocation is done before the program starts taking memory only on compile time.

Disadvantage

1. not highly scalable
2. offers limited memory flexibility
3. size of data must be known

Scanned with OKEN Scanner

Stack storage allocation :-

* used for memory allocation of local variables of a function (including main), where memory is allocated at when the function is called.

* memory is assigned in LIFO manner, memory is allocated when a function is called & deallocated when the function returns.

Eg:- In C/C++, the local variables of a function are stored in the stack. Each time a function is called, a new "activation record" (stack frame) is pushed onto the stack to store the local variables.

once the function completes, the stack frame is popped off, & the memory used is released.

// when we call the sum function below, memory // will be allocated for the variables, a, b & ans

```
void sum(int a, int b) { int ans = a+b; cout<<ans;
```

dvantage

1. very fast memory access
2. memory automatically managed
3. memory deallocated when function completes
4. Allocation & deallocation is simple.

disadvantage

1. Stack has limited size
2. can't handle large memory.

3. Heap Storage allocation

* Another form of dynamic memory allocation, but unlike the stack, it allows memory to be allocated at runtime for variables that can grow or shrink during execution.

* the heap is used for memory that is not automatically deallocated when a function call ends.

0.3

* This type of memory allocation is used to stores objects or data that are not tied to a specific function scope.

Eg:- In c/c++, functions like malloc() or new are used to allocate memory in the heap, & functions like free() or delete are used to deallocate the memory.

```
int* ans = new int[5];
```

Advantage

1. we can retain the values of variables even if the activation records end.
2. flexible allocation

Disadvantage

1. Heap is slow compare to stack
2. there is a chance of memory leak.

24

Scanned with OKEN Scanner